

Reviewer Pack — Minimal Rank of Quadratic Forms Bounds XOR-AND Tree Width

Entry 5d0f90a4ffea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 19:32:03 UTC

Minimal Rank of Quadratic Forms Bounds XOR-AND Tree Width

Entry ID: 5d0f90a4ffea **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 19:32:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory (Quadratic Forms) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any AC0 circuit C computing the XOR-AND function on n inputs, there exists a non-degenerate quadratic form $q(x_1, \dots, x_n)$ such that the rank of q is at least $\log_2(\text{size}(C)) + \varepsilon$ for some constant $\varepsilon > 0$.’, ‘Equivalently, for any instance F of the XOR-AND problem with m clauses and n variables, the minimal rank of any non-degenerate quadratic form representing the conflict set of F is at least $\log_2(m) + \varepsilon$.’]

Rationale (proposer’s reasoning):

[‘Quadratic forms have been used in studying circuit complexity before, but typically in the context of proving lower bounds on circuit size. This conjecture proposes a connection between the rank of quadratic forms and XOR-AND tree width, which could potentially provide new insights into the structure of AC0 circuits.’, ‘If true, this would suggest that there is a deep connection between algebraic number theory and complexity theory, particularly in the context of understanding the limitations of AC0 circuits.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6a4fa561bc637509

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given AC0 circuit C of size up to 40 inputs, the minimal rank of any non-degenerate quadratic form representing its output is at least $\log_2(\text{size}(C)) + \varepsilon$ if and only if the minimum rank across all generated forms for 30 different seeds is greater than or equal to $\log_2(\text{size}(C)) + \varepsilon$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            # Find pivot
            max_row = i
            for j in range(i + 1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]

            # Eliminate below
            for j in range(i + 1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

        rank = sum(1 for row in A if any(row))
        return rank

    def log2(x):
        return math.log2(x) if x > 0 else float('-inf')

    def xor_and_tree_width(n):
        # Simplified heuristic for XOR-AND tree width
        return n // 2 + 1

    def generate_ac0_circuit(n):
```

```

# Generate a random AC0 circuit of size n
circuit = []
for _ in range(2**(n-1)):
    gate = random.choice(['XOR', 'AND'])
    inputs = [random.randint(0, 1) for _ in range(n)]
    circuit.append((gate, inputs))
return circuit

def evaluate_circuit(circuit):
    # Evaluate the AC0 circuit on a random input
    n = len(circuit[0][1])
    input_ = [random.randint(0, 1) for _ in range(n)]
    result = input_
    for gate, inputs in circuit:
        if gate == 'XOR':
            result = [a ^ b for a, b in zip(result, inputs)]
        elif gate == 'AND':
            result = [a & b for a, b in zip(result, inputs)]
    return result

def generate_quadratic_form(n):
    # Generate a non-degenerate quadratic form
    Q = [[random.randint(0, 1) if i == j else 0 for j in range(n)] for i in range(n)]
    return Q

def rank_of_quadratic_form(Q):
    # Compute the rank of the quadratic form using Gaussian elimination
    A = [row + [1] for row in Q]
    rank = gaussian_elimination(A)
    return rank

n = random.randint(5, 40)
circuit = generate_ac0_circuit(n)
tree_width = xor_and_tree_width(n)
epsilon = 1e-6
min_rank = float('inf')

for _ in range(30):
    input_ = evaluate_circuit(circuit)
    Q = generate_quadratic_form(n)
    rank = rank_of_quadratic_form(Q)
    if rank < min_rank:
        min_rank = rank

conjecture_holds = min_rank >= log2(tree_width) + epsilon
counterexample = "" if conjecture_holds else f"min_rank={min_rank}, tree_width={tree_width}"

return {
    "metric_name": "min_rank",
    "metric_value": min_rank,
    "instances_tested": 30,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"min_rank < log2(tree_width) + epsilon\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition was not unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12938
2	propose	ollama_remote	glm4:latest	0	0	14897
3	preregistration	ollama_remote	glm4:latest	0	0	5692
4	novelty	ollama_remote	glm4:latest	0	0	4761
5	novelty	ollama_remote	glm4:latest	0	0	5399
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14029

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11081
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11811
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12162
10	judge	ollama_remote	glm4:latest	0	0	18915

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111684 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/5d0f90a4ffea.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5d0f90a4ffea.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5d0f90a4ffea.tar.gz (if generated)